Beating a greedy Catan bot with a better evaluation, not deeper search

Team bakarwadi · 2-player Catan hackathon, 17 October 2026 · submitted agent: `bot_cop1fix`

1 The problem and the budget

We had to build a bot for 2-player Catan, without trades between players, on the catanatron engine. It plays against F (catanatron's `ValueFunctionPlayer`), against O (the organisers' trained bot) and against the other teams. The only thing hidden from us is the opponent's unplayed development cards. Their resource hand we can work out exactly: there are 19 cards of each resource, so their hand is 19 minus the bank minus ours. The clock gives about 300 ms plus 12.5 ms of CPU per decision, so roughly one second for a whole game.

Before deciding anything, we timed the basic operations. Copying the game and playing one move costs 0.10 ms, and scoring a position costs 0.09 ms. Playing one random game to the end costs 11 ms. So in one decision we can try maybe 60 to 100 moves, and we cannot afford even one random playout. This killed MCTS for us straight away, and deep alpha-beta also.

We still checked the search route on catanatron's own AlphaBeta bot, which uses F's scoring. At depth 2 with no clock it beat F only 57.5 % of the time in 40 games. Its same-turn version won 8 % under the clock, because it keeps running out of time. So we decided early that our time should go into scoring positions well, and that any search would be very shallow.

2 First we studied F

We read F's code. F scores a position as public VPs times $3{\cdot}10^{14}$, plus production times 10^{8}, plus everything else at weights of at most 10^{4}. In practice this is a strict priority order: F grabs any visible point immediately, and production only breaks ties. F also counts only *public* points, so a hidden VP card or a knight still in hand is worth almost nothing to it.

We ran 60 F-vs-F games to see what this does to its play. The winner held Largest Army in only 3 % of games. Players bought 2.3 development cards per game, and 17 cards per game were lost to sevens, because F hoards. That told us where the free points were. F never fights for Largest Army (2 VP) and does not care about VP cards. Our v5 bot ended up holding Largest Army in about 89 % of its wins against F, with about 1.5 hidden VP cards per win.

3 How our bot works

3.1 The scoring formula

The bot judges a position with one formula. It is our score minus a fraction of the opponent's score, plus small bonuses for being ahead in the knight race and the road race:

$$V(s) = S_{\text{me}}(s) - k\,S_{\text{opp}}(s) + w_A\,\text{clip}(K_{\text{me}} - K_{\text{opp}}, \pm 2) + \tfrac{1}{2} w_L\,\text{clip}(L_{\text{me}} - L_{\text{opp}}, \pm 2)$$

Here K is knights played and L is road length. Each player's score S adds up a few simple terms:

- points: our true VPs; for the opponent, public VPs plus 0.2 for each unknown dev card, since 5 of the 25 cards are VP cards;
- production in pips per resource, with the robber's tile counted as zero. It is worth less as the game goes on, because fewer turns are left to collect;
- how many different resources we produce, the ports we hold, and how many good empty spots our roads can reach;
- our hand: what the cards are worth, a penalty above 7 cards, and a bonus when we hold exactly what a build needs;
- development cards in hand and knights already played.

We wrote every weight in victory points on purpose. A hand that can build a city right now is worth 0.47 VP, for example. That way we could look at any weight and ask whether it made sense. A state that is only a step towards a +1 VP build should never be worth more than 1 VP. This simple check later caught a big mistake (Section 6). The opponent weight came out at $k = 0.74$, below 1. Hurting the opponent by a point is worth less to us than gaining a point ourselves, so the bot races instead of blocking.

A good human player expands first and upgrades later. We gave the bot a number φ between 0 and 1 for how far into this shift each player is:

$$\varphi = \max\Big(\text{clip}\big(\tfrac{\text{VP}-4}{4}\big),\ 1 - \min\big(1,\ \tfrac{\text{best open spot pips}}{10}\big)\Big)$$

When φ is near 0, wood and brick count for more, for roads and settlements. Near 1, wheat and ore count for more (×1.2–1.3), for cities and dev cards. The expansion terms fade by $\mathbf{1 - 0.35\varphi}$, and dev cards gain by $\mathbf{1 + 0.25\varphi}$.

3.2 Choosing a move

For each decision, we sample one full game state that agrees with what we can see. We play every legal move on a copy of it, score the result and pick the best. Four things on top of this made a real difference:

1. **Averaging luck.** When the simulator buys a dev card, it draws one random card. If that card happens to be a VP card, buying looks brilliant; if it is a knight, buying looks average. The decision becomes a coin toss, so we use the average value of the cards still unseen, about 0.30 VP. A robber steal we count as one card gained and one card lost, not as the card the simulator happened to pick.
2. **Looking one step past moves that pay later.** A 4:1 trade makes our hand smaller, so on its own it scores lower. So we score a trade by the best build it allows right after. A knight is scored together with its robber move, and Road Building with its two free roads.
3. **Planning the rest of the turn.** We take the top 3 moves and play each one forward greedily until the turn ends. We switch away from the normal choice only if a plan wins by more than 0.05 VP, because these plans are noisier.
4. **Staying inside the clock.** As CPU runs low, the bot drops the turn plan, then the look-ahead step. Below 0.1 s left it stops simulating and follows a fixed order: city, settlement, knight, dev card.

3.3 A learned opening

In 2-player Catan, the four starting settlements decide most of a player's production for the whole game, and the first decision gets 300 ms. So this was a good place to use learning. We fit a logistic regression for the chance of winning given the four settlements, with 17 weights:

$$P(\text{win}) = \sigma\Big(\textstyle\sum_i w_i\,(x_i^{\text{me}} - x_i^{\text{opp}}) \pm w_{\text{seat}}\Big)$$

We generated 16,000 games of our bot against itself. In half of the placements we picked the settlement at random from the ten richest free spots, so that the model sees openings our bot would never choose. On held-out games, the model reached an AUC of 0.74 and a log-loss of 0.600; knowing only the seat gives 0.693. Gradient boosting did no better, so we kept the linear model.

The biggest weights were for the number of distinct resources (+0.28), a 3:1 port (+0.24), min(wheat, ore) pips (+0.17) and total pips (+0.16). Balance matters more than raw production. With nobody to trade with, a missing resource hurts every turn.

The placement order is P1, P2, P2, P1. At each of our picks, the bot tries its best candidate spots. For each candidate it predicts the opponent's next pick with our older hand-written formula, finishes the draft, and scores the final four settlements with the model. A CPU deadline stops this search well before the clock.

4 How we tuned and tested

We tuned the weights with SPSA, by self-play only. Win rate has no gradient, and SPSA needs only two bots per step, however many weights there are. We keep the weights as logs, $\theta_k = \theta_k^0 e^{x_k}$, so every nudge is a percentage. Each step pushes all weights at once in a random direction $\Delta \in \{\pm 1\}^d$. One bot gets $x + 0.25\Delta$ and the other $x - 0.25\Delta$, and they play 64 paired games against each other. If s is the "+" bot's win rate, we update $x \leftarrow x + a_t(2s - 1)\Delta/2$. 93 steps took about 12 minutes on 4 cores.

We kept F out of all tuning and all choosing. We never trained against it and never picked a version because it did better against F. It was only the final measuring stick, like a test set. To choose between versions, we played them head to head on the same boards and dice, with seats swapped. This removes board luck, so we need far fewer games. We did not believe a change until it had at least 300 games and about 2 standard errors.

5 What we tried, and what happened

All the F numbers below come from the official harness, with the clock on, seats alternating and fixed seeds. The 95 % interval is $\pm 1.96\sqrt{p(1-p)/n}$. "h2h" is the head-to-head win rate against the previous version.

Version	What changed	vs F	n	h2h vs parent
v1	scoring formula + trade-then-build	0.635 ± 0.067	200	
v2	+ knight and road-building follow-up	0.683 ± 0.046	400	
v3	+ opening that looks at the opponent's picks	0.726 ± 0.028	1000	0.537 ± 0.056
v5	+ SPSA weights (our first submission)	0.791 ± 0.025	1000	0.550 ± 0.049
combo	v5 + regime φ + turn plan	0.787 ± 0.025	1000	0.524 ± 0.018
op1	v5 + learned opening	0.786 ± 0.020	1600	0.543 ± 0.040
comboop1	**combo + learned opening**	**0.820 ± 0.024**	**1000**	**0.523 ± 0.031**
cop1fix	+ Monopoly after rolling; turn plan only with ≥ 0.9 s left	0.770 ± 0.041	400	0.503 ± 0.035*

Table 1. Our main line. For combo and op1 the parent is v5; for comboop1 it is combo. On the same 1000 seeds, comboop1 beat F 0.820 and combo 0.787. On op1's 1600 games, v5 scored 0.761. *With the Monopoly fix only, 800 games.

The early climb. v1 was only the formula plus "trade, then build", and it already beat F 63 % of the time. In v1 games we noticed that the bot finished games still holding knights. Playing a knight looked like "losing a card", because the gain only comes from the robber move after it. Adding that follow-up step gave about +5 points (v2). Next we made the opening think about the snake order, scoring each pick as our best pair minus the opponent's best pair, and that gave about +4 (v3). Then SPSA gave +6.5, the biggest single jump (v5).

The weights SPSA moved are interesting in themselves. A knight in hand fell from 0.25 to 0.09 VP, so play them and don't keep them. Production per pip halved (0.11 to 0.055) while wheat and ore went up ×1.6. The opponent weight fell from 0.9 to 0.74, and diversity rose from 0.12 to 0.27.

Leaderboard. v5 scored 0.779 ± 0.026 against F and 0.626 ± 0.030 against O. comboop1 then scored 0.813 ± 0.024 against F and 0.658 ± 0.029 against O, at 277 ms mean CPU per game, with 0 games over the clock and 0 faults. Our own F estimates were 0.791 and 0.820, both inside the leaderboard intervals, so our testing setup was telling the truth. [TODO: leaderboard numbers for bot_cop1fix.]

Then we hit a plateau. After v5, almost everything we tried came out level or worse:

Idea	h2h vs v5	n
Learning the whole formula by logistic regression on game outcomes	0.10–0.25	≥ 300
Before ending the turn, averaging over all the opponent's dice rolls	0.42–0.46	400
Extra reward for being "close to" a build (vs v2)	0.417 ± 0.056	300
Endgame "danger mode"; robber look-ahead	0.497–0.498	600
A second SPSA round; SPSA with the regime switched on	0.503	600–1200
1v1 tips from the Catan community (hold knights, block bottlenecks, …)	0.47–0.51	600 each
A bandit that learns which of 4 styles works against each opponent	0.530	400

Table 2. Ideas we tried after v5. The bandit also scored 0.805 against F, but at 400 games we could not tell it apart from comboop1.

Some of these surprised us. Averaging over the opponent's dice made the bot worse, not better. We think the reason is that our formula was tuned on positions at the end of our own turn, and here we were scoring positions in the middle of the opponent's turn. The "close to a build" reward made the bot sit on its cards instead of building, and it then lost them to sevens.

The regime and the turn plan were each only a small gain, about +1.7 standard errors. Together, in combo, they first measured 0.508 over 1200 games, so the gains did not simply add. Only after 3000 games did combo show 0.524. The turn plan also taught us a lesson: our first version scored the end of the turn on the one random dev card it drew, and it lost to v5 (0.46). Once we scored the draw by its average, it won 0.51–0.53. What finally

moved us forward again was the learned opening, which brings in information the formula does not have: op1 gained 2.5 to 5 points against every bot we tried.

Checking single decisions. Win rates average everything together, so a bad habit in a rare situation stays hidden. We checked about 950 decisions from real v5 games. For each possible move we played 24 random continuations, with the same dice sequences for every move. Then we measured how much better the best alternative was than the move the bot chose. Half the rollouts picked the best alternative and the other half measured it, because the best of several noisy numbers always looks too good.

Almost every kind of decision came out close to zero. One was clearly bad: **playing Monopoly before rolling cost about 7.3 ± 5.7 points**. Rolling first is free information. Playing Monopoly first also fills the hand just before a possible 7. We fixed it, and the fix did no harm (0.503 against comboop1 over 800 games, as the case is rare).

Trying to break our own bot. In a separate session we attacked comboop1 with three adversarial bots over 700 clocked games. There were no crashes and no exploit. But 2 of 500 very long games ran out of clock: the turn plan had no deadline and took up to 420 ms on one move. In the submitted bot the turn plan runs only with at least 0.9 s left, and mean CPU fell from 427 to 300 ms per game. Against F it scored 0.770 ± 0.041 in 400 games, lower than comboop1's 0.820 and close to 2 standard errors. So this fix may have cost us some strength. We chose to stay safe on the clock.

6 What we learned

1. **A model that predicts the winner well is not a model that chooses moves well.** This was our biggest lesson. We fit logistic regression to predict the winner from our own features. It was 78 % accurate on held-out games, but as a bot it won only 10 % against our hand-tuned one. When we looked at the weights, "holding the cards for a city" was worth +1.48 VP. Players holding those cards are usually the ones winning, because winners have more resources. The model learned that correlation. The greedy bot then compares "keep holding" (+1.48) with "build the city" (+1 VP plus production, minus the 1.48). Holding looks better, so the bot hoards and never builds. Played knights even got a negative weight, because they move together with army lead. Fitting only on positions right after our moves still gave 0.19. More outcome data will not fix this. The fix needs a signal about decisions: compare the chosen move with the moves not chosen, or use TD learning.
2. **The same kind of model worked for the opening, and we think we know why.** When placing settlements, all four are compared at one moment, and nothing is half-built. Predicting who wins from a placement and choosing the placement are the same question. In the middle game they are not.
3. **With a tight compute budget, a better evaluation beats deeper search.** Better scoring took us from 50 % to 79 % against F at the same CPU cost. Search on F's scoring gave 57.5 %, and none of our search add-ons cleared 2 standard errors.
4. **Look at what the opponent's goal ignores.** F counts public points only, so Largest Army and VP cards were nearly free points.
5. **Once self-play has tuned the weights, small new terms get absorbed.** A second SPSA round, the regime and the race weights all stayed within 2 standard errors of v5. What still helped was new information, like the learned opening.
6. **Measuring carefully was half the work.** SPSA's own log stays near 50 % by design, because its two bots are almost the same, so progress must be measured against a frozen older version. At 400 games the error bar is ±4.4 points, and several "improvements" we saw at 40–60 games disappeared at 300. Decision checks fool you the same way: a robber signal of +2.9 at 35 decisions vanished at 247.

7 What we would do next

The bot guesses the hidden cards only once per decision and leaves about 80 % of the clock unused. Averaging over several guesses would use that time. The opening assumes the opponent drafts like our old formula, trains only on self-play, and ignores roads. The proper fix for lesson 1 is TD(λ) learning on positions right after our moves. We would also re-tune the weights with the turn plan switched on.

References

1. B. Collazo. *catanatron*: Settlers of Catan simulator and bots. github.com/bcollazo/catanatron (GPL-3.0).

2. J. C. Spall. Multivariate stochastic approximation using a simultaneous perturbation gradient approximation. *IEEE Trans. Automatic Control*, 37(3), 1992.
3. M. Buro. Improving heuristic mini-max search by supervised learning. *Artificial Intelligence*, 134, 2002.
4. G. Tesauro. Temporal difference learning and TD-Gammon. *Communications of the ACM*, 38(3), 1995.
5. I. Szita, G. Chaslot, P. Spronck. Monte-Carlo tree search in Settlers of Catan. *Advances in Computer Games*, 2010.